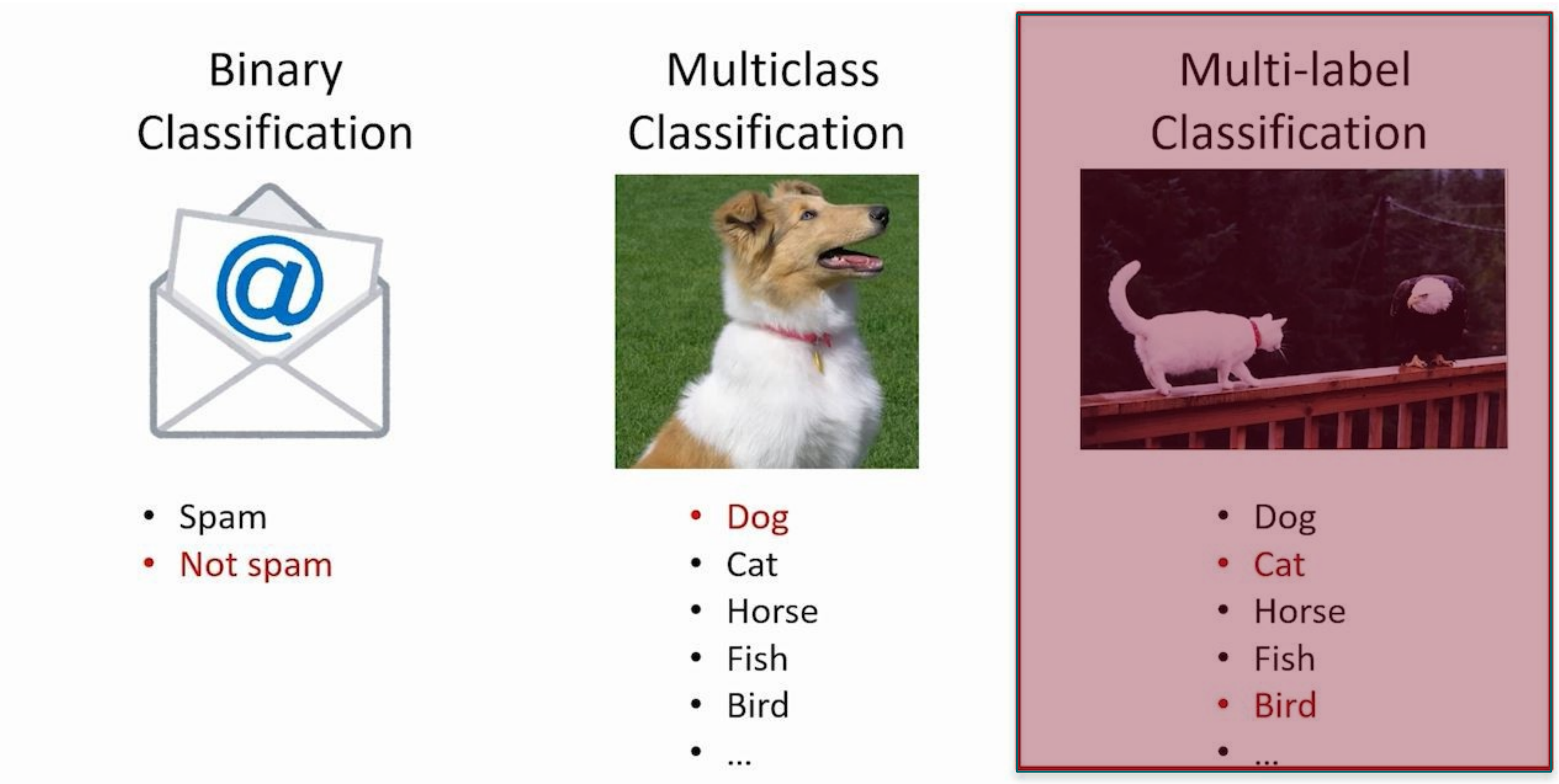
Module 2

Multi-Label Classification

Multi-Label Classification

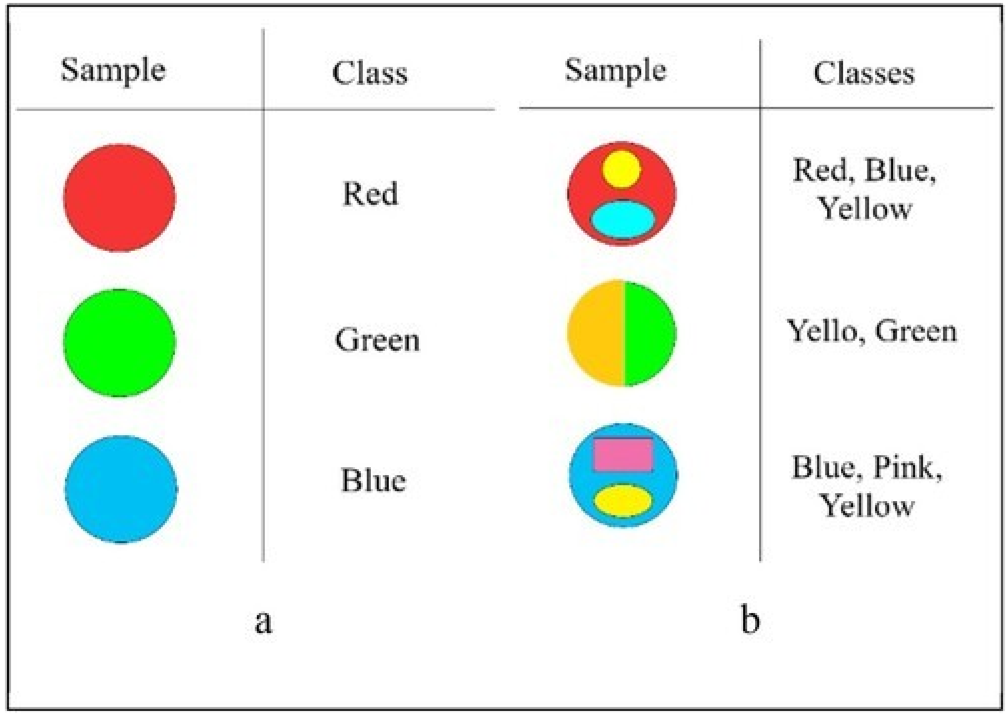
Types of Classification Problems

- The 3 main classification problems are:

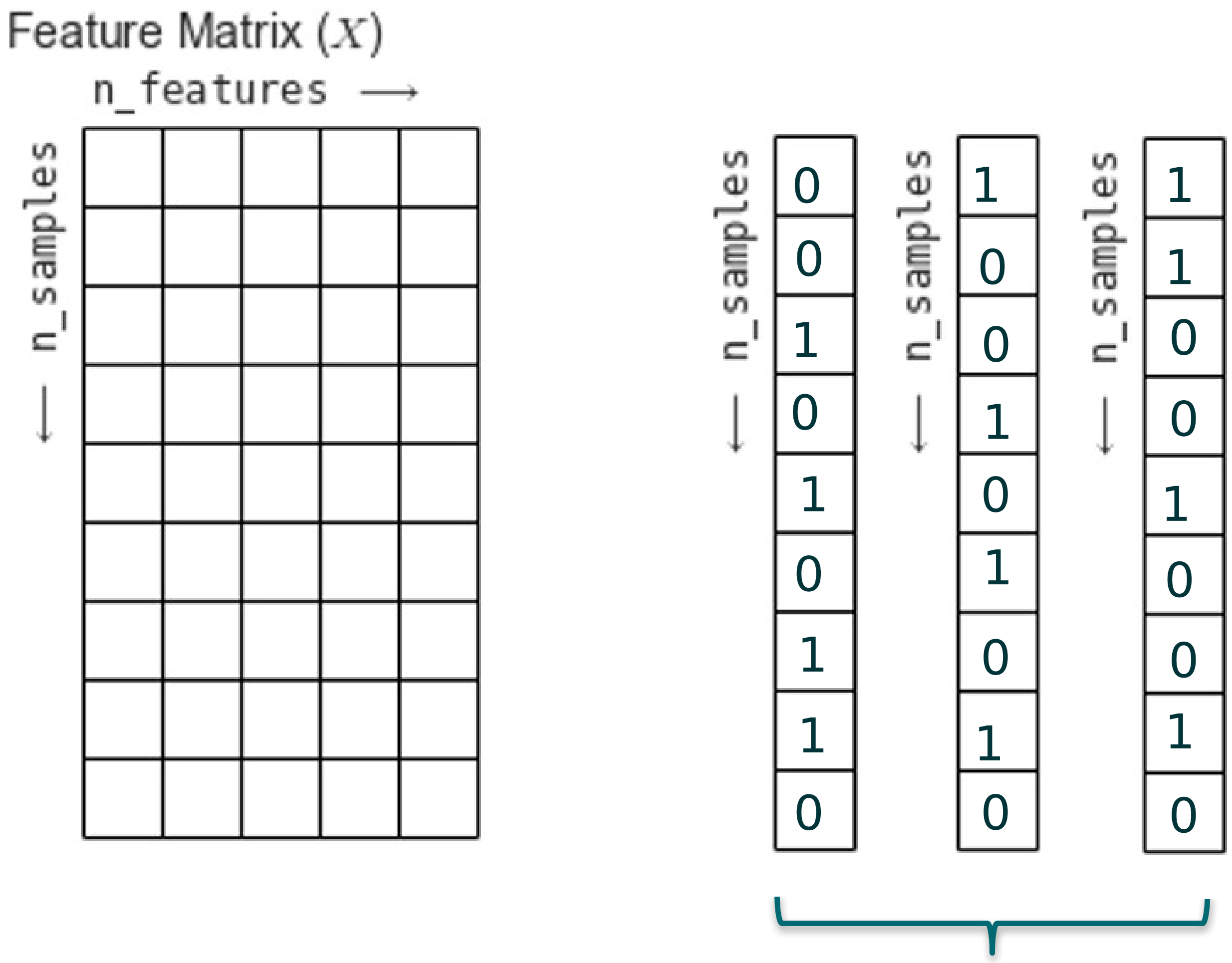


Multi-Label Classification

- is the supervised learning problem where an instance may be associated with multiple labels



Multi-Label Classification



Response matrix

Binary Relevance

- Decomposes learning tasks into independent binary problems.
- Returns a propensity/prediction vector for each response
- We fit each response independently

X	Y_1	Y_2	Y_3	Y_4
$X^{(1)}$	0	0	0	1
$X^{(2)}$	1	0	0	0
$X^{(3)}$	1	0	0	1
$X^{(4)}$	0	1	0	0
$X^{(5)}$	0	1	1	0

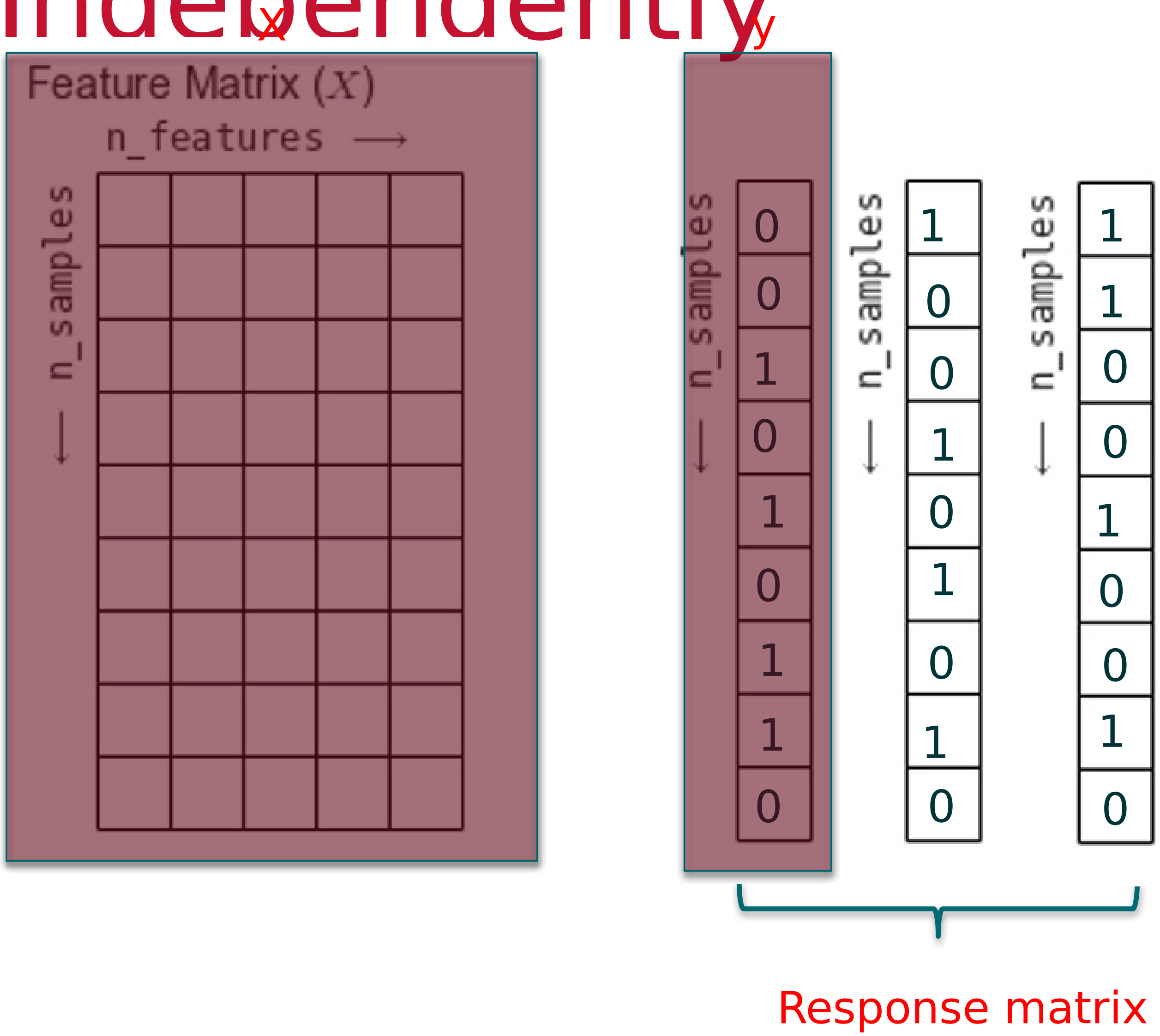
$\rightarrow$

X	Y_1
$X^{(1)}$	0
$X^{(2)}$	1
$X^{(3)}$	1
$X^{(4)}$	0
$X^{(5)}$	0

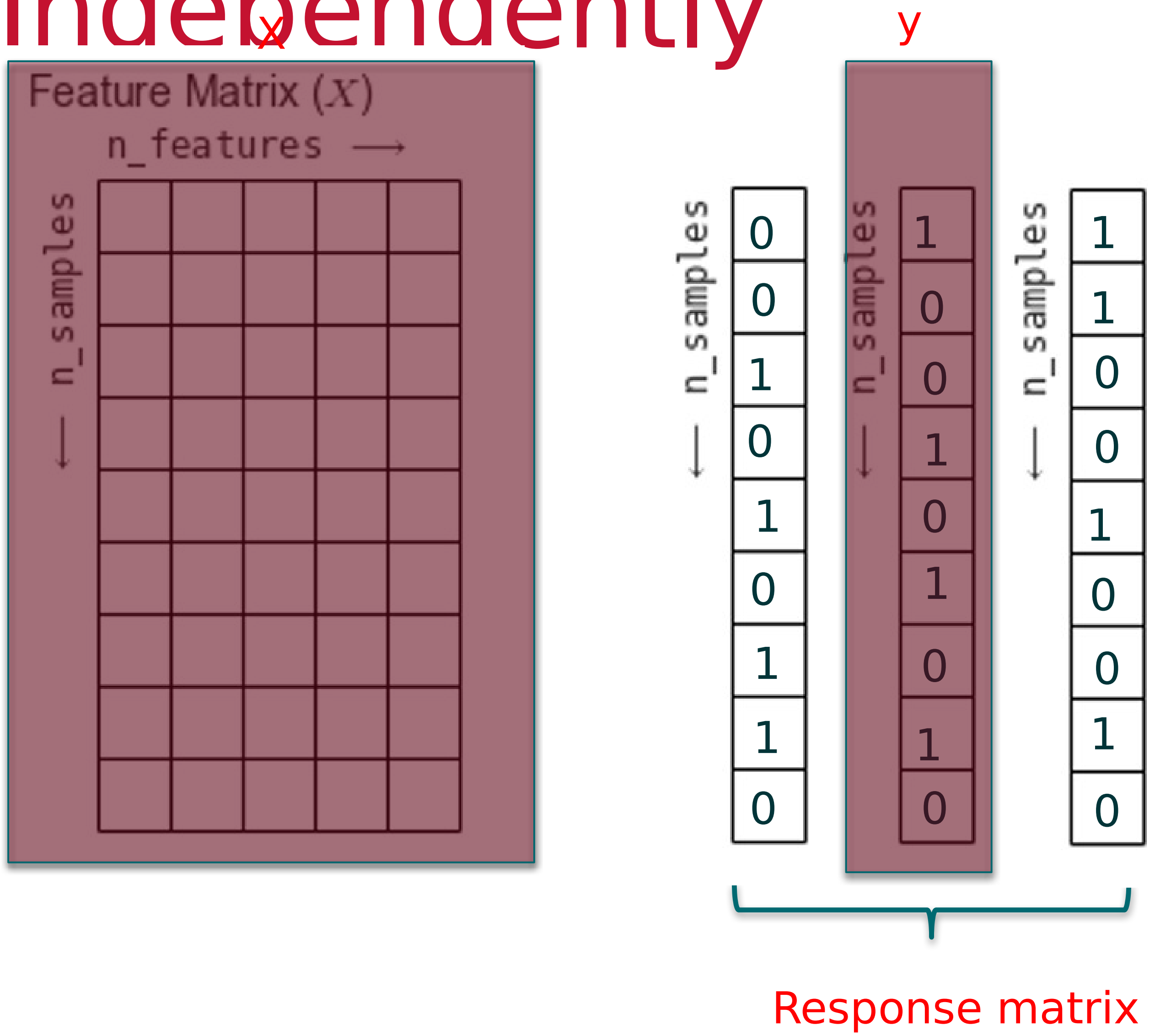
X	Y_2
$X^{(1)}$	0
$X^{(2)}$	0
$X^{(3)}$	0
$X^{(4)}$	1
$X^{(5)}$	1

X	Y_3
$X^{(1)}$	0
$X^{(2)}$	0
$X^{(3)}$	0
$X^{(4)}$	0
$X^{(5)}$	1

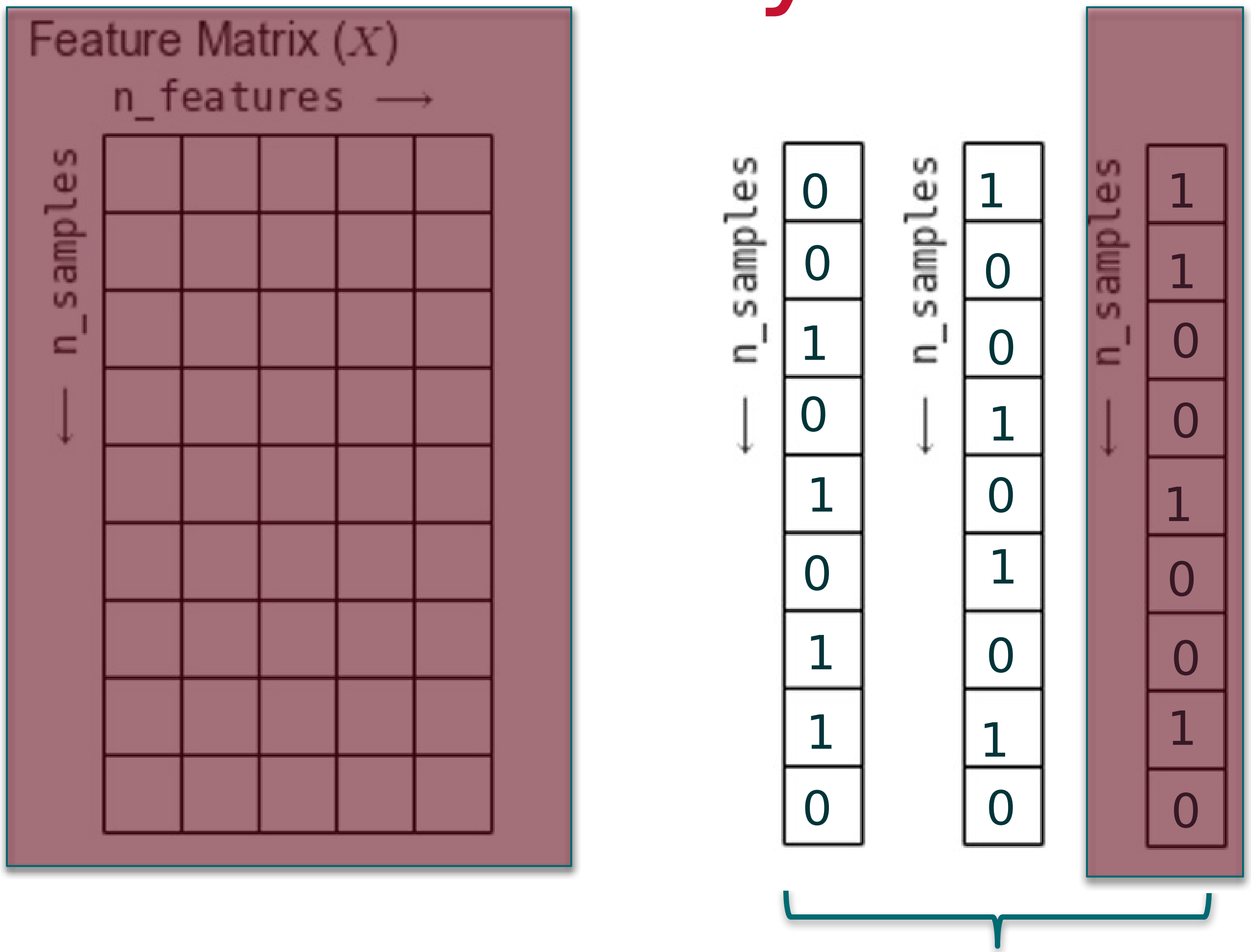
Fitting Each Response Independently



Fitting Each Response Independently



Fitting Each Response Independently



Python

Classifier Chains

- Arrange binary classifiers into a chain adding response vector to features.

X	Y_1	Y_2	Y_3	Y_4
X_1	0	1	0	0
X_2	0	1	1	0
X_3	1	0	0	0
X_4	0	1	0	0

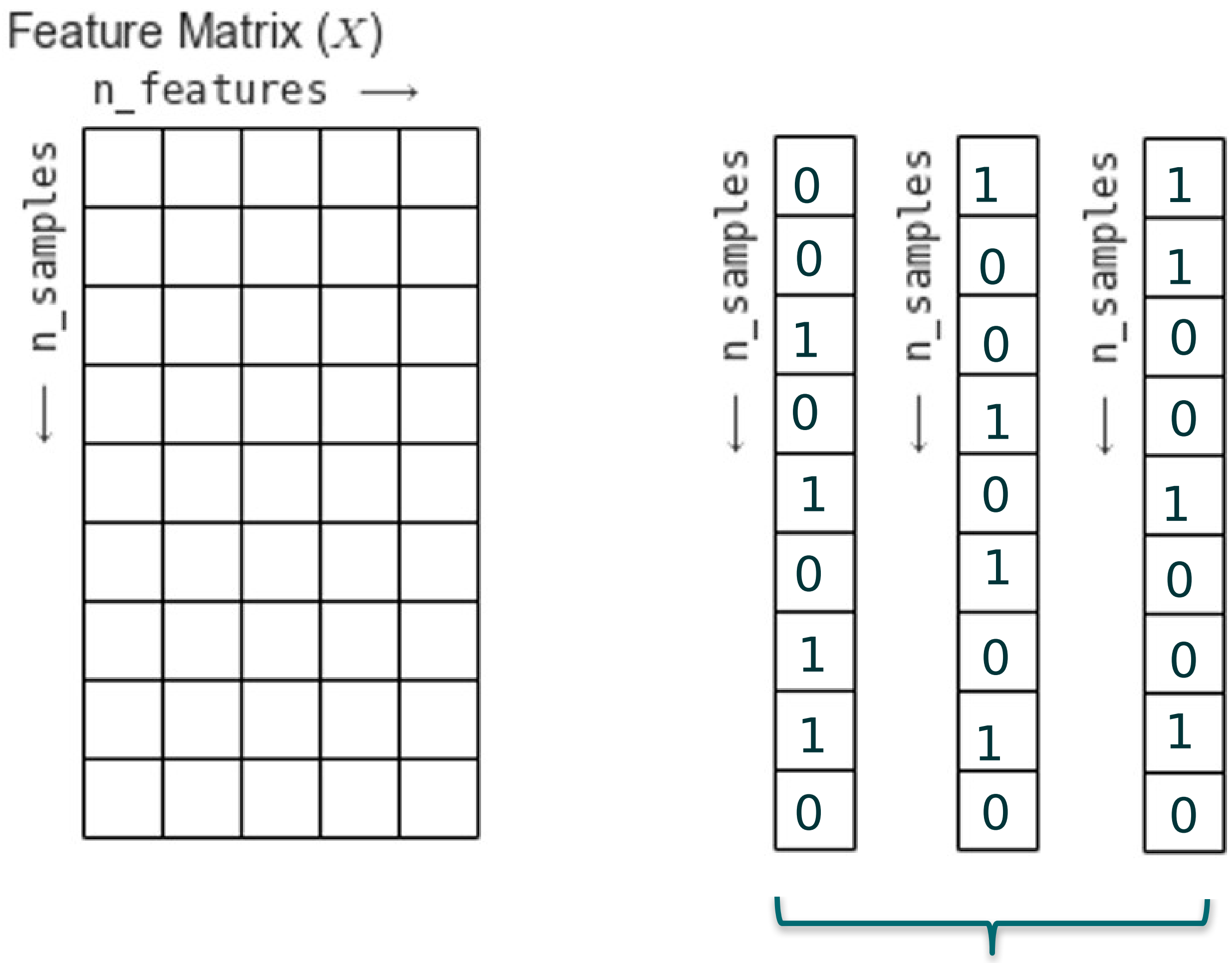
X	Y
X_1	0
X_2	0
X_3	1
X_4	0

X	Y_1	Y_2
X_1	0	1
X_2	0	1
X_3	1	0
X_4	0	1

X	Y_1	Y_2	Y_3
X_1	0	1	0
X_2	0	1	1
X_3	1	0	0
X_4	0	1	0

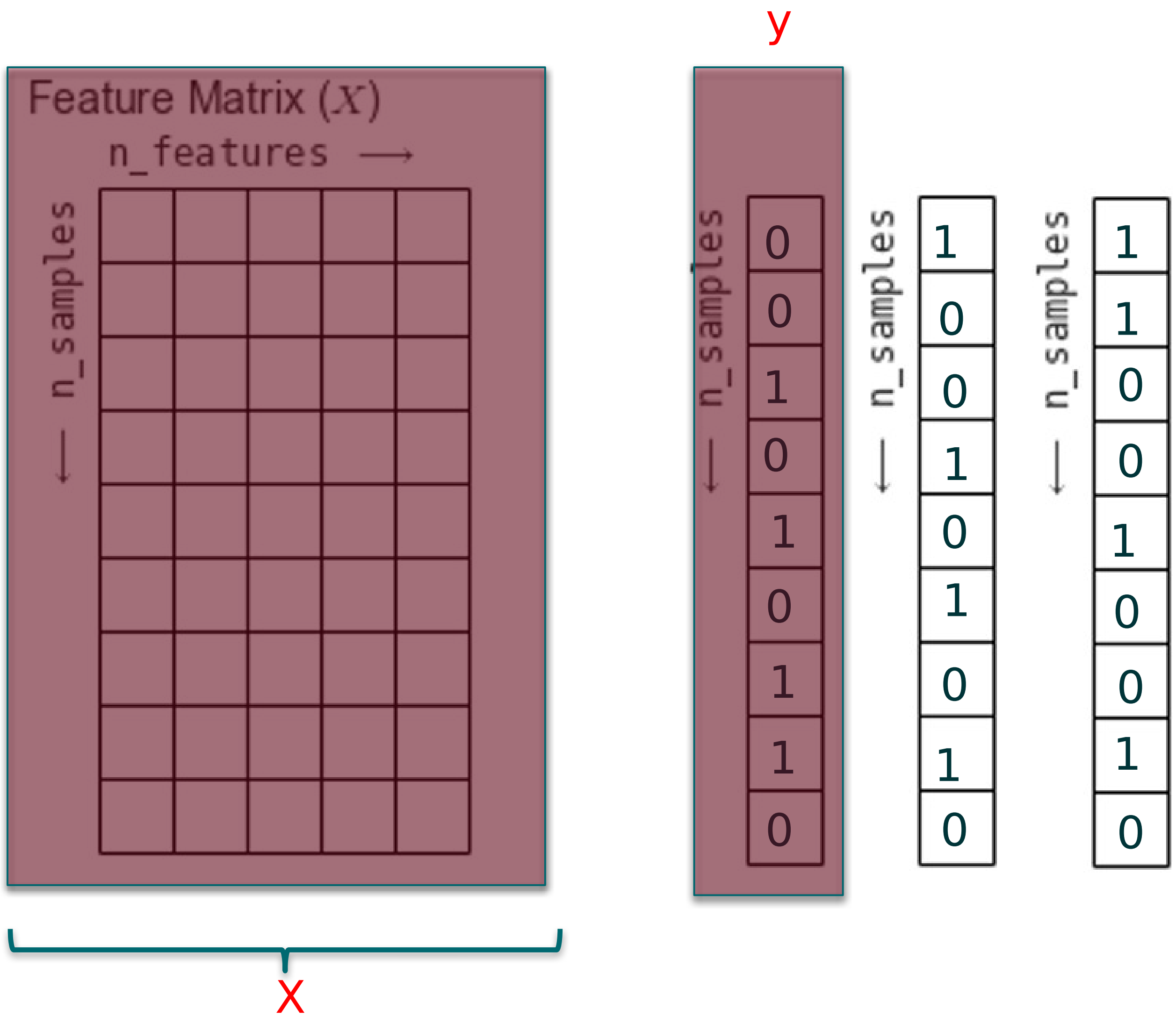
X	Y_1	Y_2	Y_3	Y_4
X_1	0	1	0	0
X_2	0	1	1	0
X_3	1	0	0	0
X_4	0	1	0	0

Multi-Label Classification

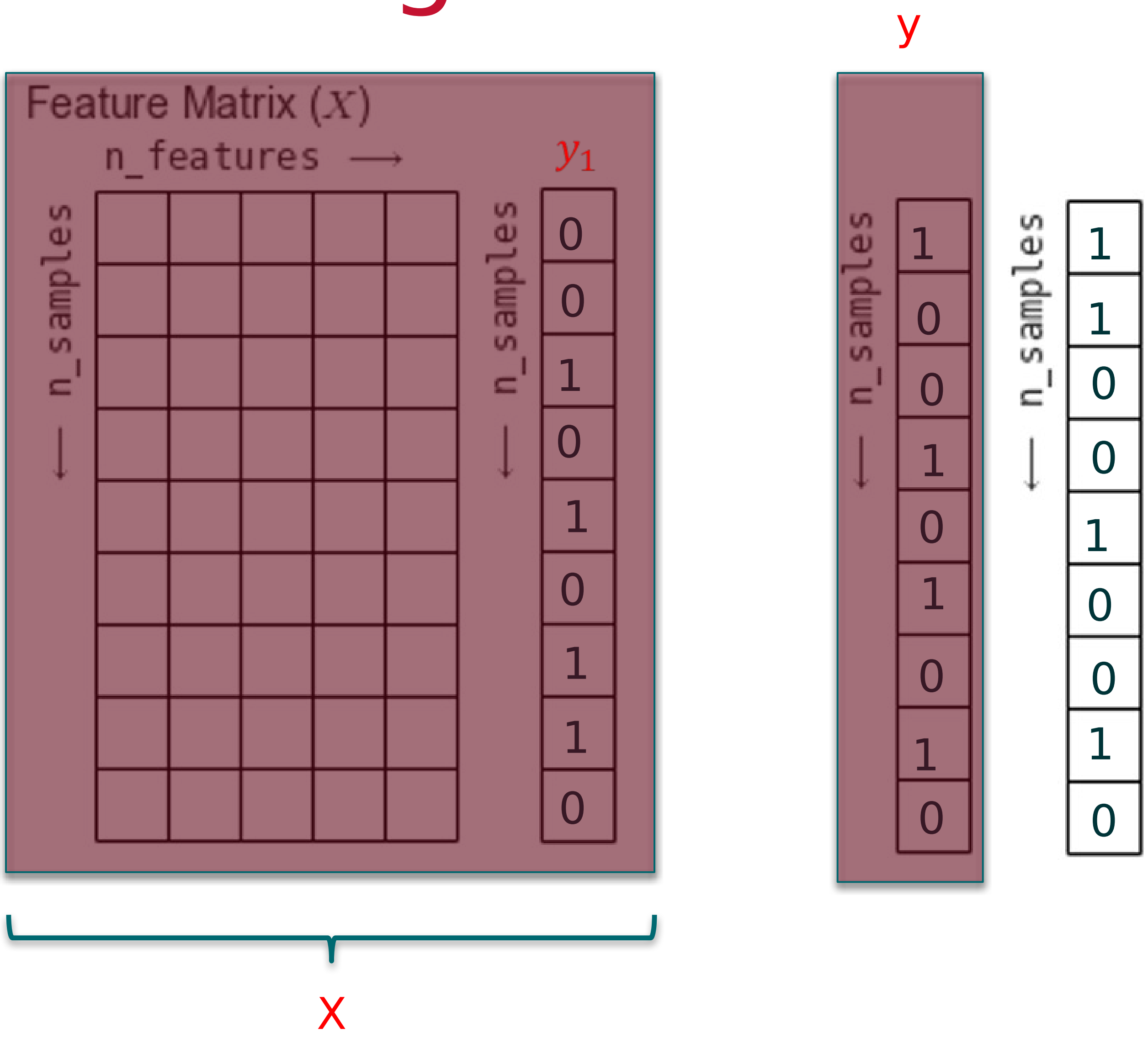


Response matrix

Creating a Chain



Creating a Chain



Python

Metrics: Exact Match Ratio

- Predictions that are exact matches to the response vectors are considered accurate
- Partial matches are considered errors

$$
\begin{pmatrix}
1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\
1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\
1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
\end{pmatrix}
=
\begin{pmatrix}
1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\
1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\
1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
\end{pmatrix}
$$

Python

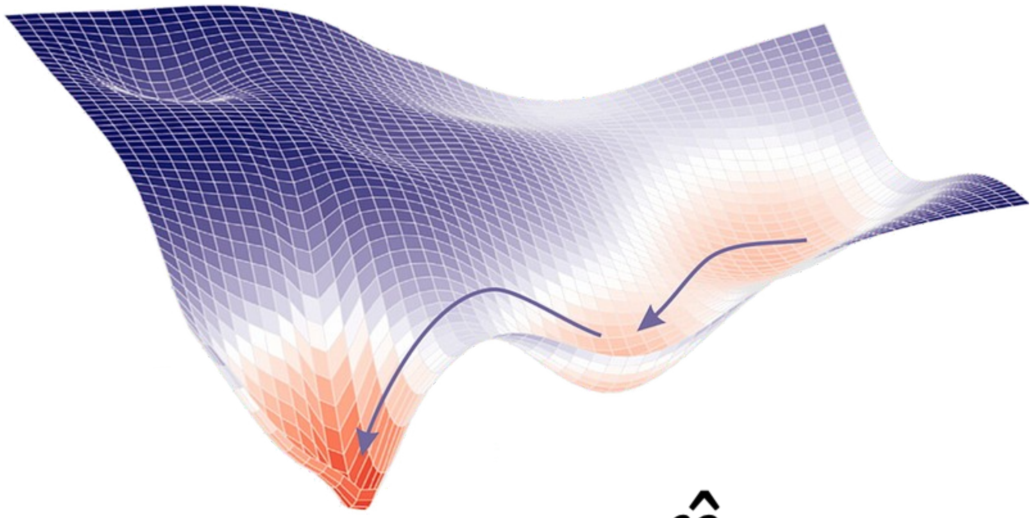
Common Cost Functions Neural Nets

- Binary Cross-Entropy
  - Loss Function

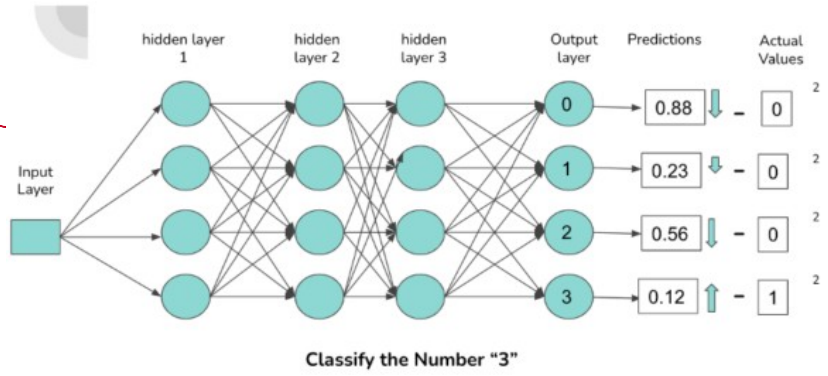
$$J(w) = -\frac{1}{n}\sum(y \log \hat{p} + (1-y)\log(1-\hat{p}))$$

  - Gradient

$$\nabla J(w) = \hat{p} - y$$

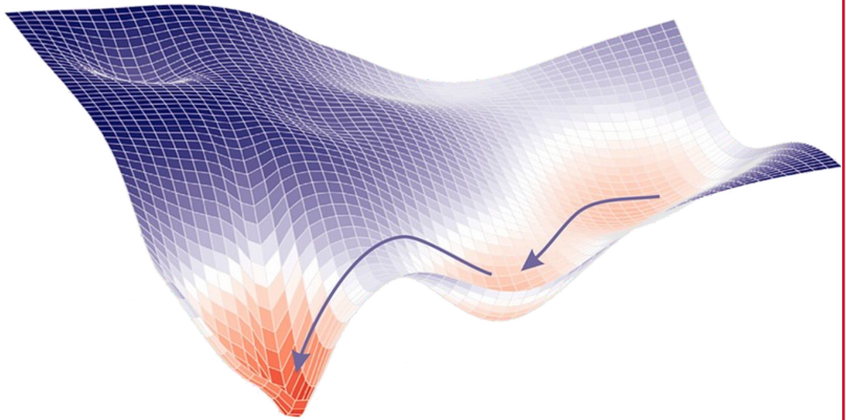


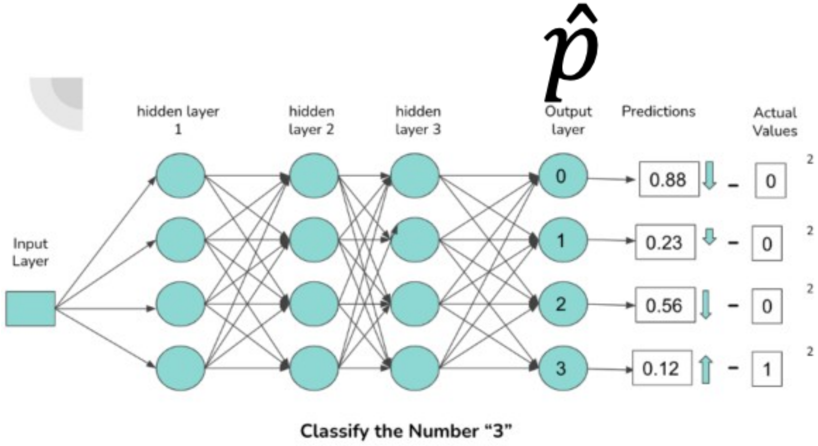
$\hat{p}$



Activation Functions Neural Nets

- Common Activations on Layers
  - ReLu
  - Tanh
  - Sigmoid
- Output Layer Activation
  - Sigmoid





Python